

UNIVERSIDAD PERUANA UNIÓN • FACULTAD DE INGENIERÍA Y ARQUITECTURA

GUÍA PRÁCTICA: KOTLIN ESENCIAL & DOMINIO KMP

Modelado de Dominio, Null-Safety, Data Classes, Corrutinas y Flow en PharmaMobile

Pregrado • VIII Ciclo • Asignatura: Desarrollo de Aplicaciones Móviles • Formato Widescreen 16:9

INTRODUCCIÓN & PROPÓSITO PRÁCTICO

Evolución del Proyecto

- > Consolidada la infraestructura base KMP, incorporamos lógica de negocio real.
- > Construcción del modelo de dominio compartido dentro de **commonMain**.
- > Aplicación de tipado seguro, inmutabilidad y gestión de estados asíncronos.

Objetivo de la Práctica

Implementar entidades de dominio robustas preparadas para Android e iOS, integrando null-safety, colecciones, corrutinas y flujos reactivos Flow.

REQUISITOS PREVIOS & MATERIALES

Hardware

PC o laptop con mínimo 8 GB de RAM y espacio libre para SDK, IDE y emuladores móviles.

Software Base

Android Studio (Koala/Jellyfish), JDK 17+, Android SDK, Git CLI y Gradle.

Línea Base Git

Proyecto **PharmaMobile** sincronizado y con repositorio local limpio (`git status`).

PASO 1: VERIFICACIÓN DE ENTORNO & GIT

Comprobación del Workspace

- > Apertura del proyecto KMP en Android Studio y verificación de sincronización Gradle.
- > Ejecución de comandos de control de versiones para asegurar trazabilidad.

Comandos de Validación

```
git status  
java -version  
git log --oneline
```


PASO 2: ENTIDAD CLIENTE & NULL-SAFETY

Contrato de Nulabilidad

- > Uso de tipos estrictos no anulables para identificadores y nombres obligatorios.
- > Uso del operador `?` para datos opcionales (ej. telefono: String?).
- > Aplicación del operador Elvis `?:` para valores por defecto seguros.

```
data class Cliente(  
    val id: Long,  
    val nombres: String,  
    val correo: String,  
    val telefono: String?  
)  
  
fun verTel(c: Cliente) = c.telefono ?: "No registrado"
```


PASO 3: PRODUCTOS, INMUTABILIDAD & COPY()

Ventajas de Data Classes

- > Generación automática de métodos auxiliares (toString, equals, hashCode).
- > Patrón **copy()** para clonar instancias modificando únicamente propiedades específicas sin mutar el objeto original.

```
data class Producto(  
    val id: Long,  
    val nombre: String,  
    val precio: Double,  
    val stock: Int  
)  
  
val upd = original.copy(stock = 90)
```


PASO 4: CONSULTAS SOBRE COLECCIONES

Filtrado (filter)

Selección de elementos bajo condiciones lógicas (ej: productos con stock disponible).

```
productos.filter { it.stock > 0 }
```

Transformación (map)

Conversión de colecciones de objetos a listas de propiedades específicas (ej: nombres).

```
productos.map { it.nombre }
```

Búsqueda (find)

Localización segura de elementos con manejo de tipos anulables resultantes.

```
productos.find { it.id == 2L }
```


PASO 5: PEDIDOS & ESTADOS CON SEALED CLASSES

Control Jerárquico de Estados

- Restricción de opciones a un conjunto cerrado evaluado de manera exhaustiva mediante **when**.
- Capacidad de asociar datos adicionales a subtipos específicos (ej. `motivo` en rechazos).

```
sealed class EstadoPedido {  
    data object Pendiente : EstadoPedido()  
    data object Procesando : EstadoPedido()  
    data class Rechazado(val motivo: String) : EstadoPedido()  
}
```


PASO 6: ASINCRONÍA CON CORRUTINAS & FLOW

Operaciones Asíncronas y Reactivas

- > Funciones **suspend** para tareas puntuales que requieren espera sin bloquear hilos.
- > Emisión de múltiples valores a lo largo del tiempo mediante **Flow**, `emit()` y `collect()`.

```
fun observarProductos(): Flow<List<Producto>> = flow {  
    emit(emptyList())  
    delay(1000)  
    emit(productosRemotos)  
}
```


MEJORES PRÁCTICAS & TROUBLESHOOTING

Mejores Prácticas

- Mantener el dominio 100% puro en `commonMain` sin acoplamiento a Android/iOS.
- Favorecer `val` frente a `var` para garantizar inmutabilidad.
- Commits atómicos frecuentes en Git.

Troubleshooting Común

- **Error de Sincronización Gradle:** Verificar conexión a internet y limpiar caché (Build → Clean).
- **Nullability Crash:** Evitar operador `!!` y usar Elvis `? :.`

CONCLUSIONES & RECURSOS ADICIONALES

Síntesis de la Sesión

- > El dominio compartido en KMP unifica la lógica empresarial garantizando consistencia.
- > Null-safety y data classes previenen errores críticos en tiempo de ejecución.
- > Corrutinas y Flow sientan las bases para arquitecturas reactivas y offline-first.

The logo for Kotlin Multiplatform (KMP) is displayed in a large, bold, blue font. The letters 'K', 'M', and 'P' are stylized and connected.

Documentación Oficial Kotlin & JetBrains Multiplatform.